

ХМЕЛЬНИЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Факультет інформаційних технологій

Кафедра комп'ютерної інженерії та інформаційних систем

Лабораторна робота № 1

з дисципліни «Кросплатформне програмування»

Базові операції у Java: змінні, методи та класи

Тривалість: 4 академічні години

Методичні матеріали для здобувачів першого (бакалаврського) рівня

Викладач: старший викладач
Денисюк Дмитро Олександрович

1. Матеріально-технічне забезпечення

Мета

Налаштувати середовище розроблення Java-проектів і реалізувати консольну програму, у якій обчислення зосереджено в предметному класі. Результат має відтворюватися засобами Maven, проходити модульні тести та зберігатися у загальнодоступному репозиторії Git.

Результати навчання

Після виконання роботи студент повинен:

1. **Знати** призначення JDK, компілятора `javac`, байткоду та JVM;
2. **Знати** роль Maven, JUnit і Git у відтворюваному процесі розроблення та перевірки;
3. **Вміти** створювати й відкривати Maven-проект в Apache NetBeans та виконувати його складання;
4. **Вміти** оголошувати змінні, обирати типи й реалізовувати методи з параметрами та значеннями, що повертаються;
5. **Вміти** описувати клас із полями, конструктором і методами, а також створювати об'єкти оператором `new`;
6. **Вміти** реалізовувати модульні тести, виконувати перевірку з чистої папки та публікувати зафіксовану версію проекту в Git.

Необхідні знання

Необхідно вміти створювати файли й каталоги, виконувати арифметичні обчислення з десятковими числами та розрізняти файл і каталог. Досвід роботи з Maven, JUnit і Git не вимагається.

Програмне забезпечення

Для цієї та наступних лабораторних робіт рекомендовано інтегроване середовище розроблення **Apache NetBeans 31**. Випуск поширюється за ліцензією Apache License 2.0, функціонує у Windows, Linux і macOS та підтримує запуск на JDK 21, 25 і 26. Середовище забезпечує редагування, складання, запуск і тестування Java-проектів, а також містить інтегровані засоби Maven і Git.

Навчальні проекти оформлюють за стандартною структурою Maven. Версію Maven фіксує Maven Wrapper, тому локальна й зовнішня перевірки виконують однакову послідовність фаз складання. Тести реалізують засобами JUnit Jupiter і зберігають у каталозі `src/test/java`.

Табл. 1: Потрібні засоби

Засіб	Вимога	Як перевірити
JDK	Eclipse Temurin JDK 25 LTS; цільовий рівень навчального коду — Java 21. Preview-функції не застосовують.	<code>java --version</code> і <code>javac --version</code>
Середовище	Apache NetBeans 31.	Help > About відображає версію NetBeans і JDK.
Складання	Maven Wrapper для Maven 3.9.16; Maven Compiler Plugin 3.15.0.	<code>mvnw.cmd --version</code> у Windows або <code>./mvnw --version</code> у Linux і macOS.
Тестування	JUnit Jupiter 6.1.3; Maven Surefire Plugin 3.5.5.	<code>mvnw.cmd -B -ntp clean verify</code> або відповідна POSIX-команда.
Контроль версій	Git 2.x і загальнодоступний віддалений репозиторій.	<code>git --version</code> , <code>git status</code> та анонімне <code>git ls-remote URL</code> .

Набір версій перевірено 1 вересня 2026 року. Apache NetBeans 31 випущено 10 серпня 2026 року. Java SE 25 має статус LTS, а параметр Maven `release=21` обмежує синтаксис, API й формат байткоду рівнем Java 21.

Організація файлів і захист даних

Вихідні файли зберігають у UTF-8. Каталог `target`, файли `.class`, журнали, кеші середовища та локальні параметри не додають до репозиторію. Заборонено публікувати паролі, токени доступу, приватні ключі й файли конфігурації із секретними значеннями. Перед кожним комітом необхідно перевіряти перелік змін командою `git status`.

2. Короткі теоретичні відомості

Теоретичні положення цього розділу визначають терміни й обмеження, безпосередньо використані під час реалізації та тестування програми.

2.1. Від вихідного коду до виконання

JDK (Java Development Kit) — комплект розробника Java. Він містить компілятор `javac`, засоби діагностики та середовище виконання. **JVM (Java Virtual Machine)** — віртуальна машина, яка завантажує та виконує байткод. Така проміжна форма дає змогу запускати той самий скомпільований код на різних операційних системах, якщо для них існує сумісна JVM.

$$\text{Main.java} \xrightarrow{\text{javac}} \text{Main.class (байткод)} \xrightarrow{\text{java}} \text{JVM} \rightarrow \text{результат програми}$$

Компілятор перевіряє синтаксис і типи та створює файли `.class`. Команда `java` запускає JVM, завантажує клас і викликає точку входу `public static void main(String[] args)`. Під час запуску вказують ім'я класу `Main`, а не файл `Main.class`.

2.2. Maven як система складання та керування проєктом

Apache Maven — засіб автоматизації складання та керування Java-проєктом. Складання охоплює не лише компіляцію вихідного коду, а й послідовне виконання перевірок, тестів, створення готового пакета та інших повторюваних операцій. Maven читає опис проєкту з файла `pom.xml`. Назва POM походить від *Project Object Model* — моделі об'єкта проєкту. У цьому файлі визначають координати проєкту, версію Java, залежності, плагіни та параметри складання.

Maven застосовує стандартну структуру каталогів. Наприклад, основний код розміщують у `src/main/java`, тести — у `src/test/java`, а результати складання — у `target`. Залежності, зокрема JUnit, Maven отримує з репозиторіїв програмних компонентів відповідно до версій, указаних у `pom.xml`. Плагіни виконують конкретні операції, наприклад компіляцію або запуск тестів.

Життєвий цикл складання — упорядкована послідовність фаз. Для цієї роботи важлива послідовність `validate` → `compile` → `test` → `package` → `verify`. Якщо викликати пізнішу фазу, Maven виконає також усі попередні фази цього циклу. Фаза `verify` перевіряє, що створений пакет відповідає заданим критеріям. Команда `clean` належить до окремого життєвого циклу та видаляє результати попереднього складання. Отже, команда `mvnw.cmd -B -ntp clean verify` у Windows спочатку очищує `target`, а потім повторно компілює, тестує, пакує й перевіряє проєкт без інтерактивних запитів і зайвого виведення перебігу завантаження.

Maven Wrapper — набір файлів `mvnw`, `mvnw.cmd` і `.mvn/wrapper`, який запускає визначену проєктом версію Maven. Wrapper не є окремою системою складання. Він дає змогу студенту й системі автоматизованої перевірки виконувати однакову команду без вимоги попередньо встановлювати Maven вручну.

Maven не є єдиним способом складання Java-проєктів. Вибір засобу залежить від структури проєкту, вимог до налаштування та наявної інфраструктури.

Табл. 2: Maven та поширені альтернативи

Засіб	Модель роботи	Доцільне застосування
Maven	Декларативний файл <code>pom.xml</code> , стандартні каталоги та визначений життєвий цикл.	Проекти, для яких важливі усталена структура, керування залежностями та відтворювана команда перевірки.
Gradle	Сценарії складання мовами Kotlin або Groovy, граф завдань, інкрементальне виконання та кешування.	Проекти, що потребують гнучкого налаштування; широко застосовується також у розробленні для Android.
Apache Ant	XML-файл із явно заданими цілями та послідовностями низькорівневих завдань.	Наявні або спеціалізовані проекти, де процес складання потрібно докладно описати вручну.
<code>javac</code> і <code>java</code>	Окремі команди компілятора та середовища виконання; системою складання не є.	Невеликі навчальні приклади без автоматичного керування залежностями, тестовим циклом і пакуванням.

У межах курсу Maven обрано через стандартизовану структуру проекту, Maven Wrapper і однакову команду `clean verify` для локальної та автоматизованої перевірки. Це навчальне рішення не означає універсальної переваги Maven над іншими засобами.

2.3. Змінна, тип, значення та вираз

Змінна — іменована ділянка стану програми, значення якої має визначений тип. Java є статично типізованою мовою: тип змінної відомий під час компіляції. **Ініціалізація** — надання першого значення. Наприклад, у `double width = 4.5`; типом є `double`, іменем — `width`, початковим значенням — `4.5`.

Табл. 3: Типи, достатні для цієї роботи

Тип	Приклад	Коли використовувати
<code>int</code>	<code>int count = 3;</code>	Цілі значення в межах приблизно від $-2,1$ млрд до $2,1$ млрд.
<code>long</code>	<code>long population = 5L;</code>	Цілі значення, для яких діапазону <code>int</code> недостатньо.
<code>double</code>	<code>double price = 59.9;</code>	Дробові обчислення навчального характеру; результат зазвичай форматується.
<code>boolean</code>	<code>boolean valid = true;</code>	Логічне значення <code>true</code> або <code>false</code> .
<code>char</code>	<code>char grade = 'A';</code>	Один символ у одинарних лапках.
<code>String</code>	<code>String name = "Room";</code>	Текст; це клас, а не примітивний тип.

Вираз поєднує значення, змінні та оператори й утворює результат, наприклад `width * height`. Оператори `+`, `-`, `*`, `/` виконують арифметику, а `<=`, `==`, `||` формують логічні умови. Дужки

роблять порядок обчислення явним. Вираз $5 / 2$ дає ціле 2, а $5.0 / 2$ — 2.5, бо тип операндів впливає на ділення.

Тип `double` зберігає наближені двійкові значення. Він придатний для цієї навчальної роботи, але не для точних фінансових розрахунків у виробничій системі; там застосовують, наприклад, `BigDecimal` і чіткі правила округлення.

2.4. Методи

Метод — іменований блок коду, що виконує одну завершену дію. Параметри описують вхідні дані, а тип перед назвою — результат. У `double area()` типом результату є `double`, параметрів немає, а оператор `return` повертає обчислене значення. Метод `void print()` виконує дію без повернення значення.

```
1 public double area() {  
2     return width * height;  
3 }
```

Назва методу за конвенцією починається дієсловом або чітко називає обчислювану властивість, використовує `lowerCamelCase` та відповідає одній відповідальності.

2.5. Клас, поля, конструктор та об'єкт

Клас — опис структури й поведінки майбутніх об'єктів. **Поле** зберігає стан об'єкта, **конструктор** надає полям початкові значення, а методи реалізують поведінку. **Об'єкт** — конкретний екземпляр класу. Вираз `new Rectangle(width, height)` виділяє пам'ять, викликає конструктор і повертає посилання на створений об'єкт.

Модифікатор `private` забороняє прямий доступ до поля з іншого класу. У цій роботі достатньо розуміти, що обчислення виконуються через публічні методи. Повне обґрунтування інкапсуляції розглядатиметься в наступній лабораторній роботі.

3. Порядок виконання роботи

Налаштування середовища розроблення

Наведену конфігурацію застосовують у всіх лабораторних роботах курсу. Для кожної роботи створюють окремий Maven-проект і окремий віддалений репозиторій Git.

1. Встановіть [Eclipse Temurin JDK 25 LTS](#). Виберіть пакет JDK для своєї операційної системи й архітектури. Після встановлення відкрийте новий термінал і виконайте:

```
java --version  
javac --version
```

Обидві команди мають вивести версію 25.x. Збережіть цей вивід для розділу звіту про середовище виконання.

2. Завантажте [Apache NetBeans 31](#) і встановіть пакет для своєї операційної системи. У меню `Help > About` перевірте номер 31 і JDK, на якому запущено середовище.

3. Відкрийте **Tools > Java Platforms** і за потреби додайте каталог Temurin 25. Переконайтеся, що JDK 25 доступний для нових Java-проектів.
4. Отримайте від викладача папку або посилання на стартовий репозиторій **Lab Assignment 1**. У NetBeans виконайте **File > Open Project** і виберіть цю папку. Не створюйте новий проект: файл `pom.xml`, Maven Wrapper і публічні тести вже входять до заготовки. Перевірте, що **Source Packages** містить `Main.java`, а **Test Packages** — `Lab1PublicTest.java`.
5. У корені заготовки виконайте `mvnw.cmd --version` у Windows або `./mvnw --version` у Linux чи macOS. Очікується Maven 3.9.16; під час першого запуску Wrapper потребує доступу до мережі.
6. Встановіть **Git 2.x** і виконайте `git --version`. Далі використовуйте термінал або меню **Team > Git** в Apache NetBeans.

Постановка задачі та очікуваний результат

Пояснений нижче клас прямокутника є референсним прикладом, який потрібно проаналізувати, але не копіювати як розв'язок індивідуального варіанта. Практичну частину студент виконує у наданій заготовці **Lab Assignment 1**: зазначає номер варіанта, реалізує предметний клас і `Main`, додає власні модульні тести та запускає надані публічні тести. Завершений проект має проходити команду `clean verify` і бути доступним у публічному Git-репозиторії за зафіксованим ідентифікатором коміту.

Стартовий проєкт і початкова структура

Стартова заготовка має таку структуру Maven. Предметний клас і власний тестовий клас студент додає відповідно до свого варіанта:

```
Lab Assignment 1/
+-- .mvn/wrapper/
+-- src/
|   +-- main/java/ua/khnu/kpp/lab1/
|   |   +-- Main.java
|   +-- test/java/ua/khnu/kpp/lab1/
|       +-- Lab1PublicTest.java
+-- docs/README.md
+-- .gitignore
+-- README.md
+-- submission.properties
+-- mvnw
+-- mvnw.cmd
+-- pom.xml
```

У файлі `submission.properties` запишіть лише номер роботи та варіанта:

```
lab=1
variant=NN
```

Файл не повинен містити прізвища, номера залікової книжки, адреси електронної пошти чи інших персональних даних. Каталог `target` створює Maven; його не додають до Git.

Публічний клас `Lab1PublicTest` читає номер варіанта з `submission.properties` і перевіряє назву предметного класу, сигнатури конструктора й методів, контрольний і незалежний допустимий набори, відхилення помилкових значень та консольний протокол. До реалізації варіанта тести очікувано завершуються невдало з повідомленням про відсутній клас або незаповнені метадані. Файл `Lab1PublicTest.java` не редагують: студент додає окремий власний клас тестів, а система приймання може підключити додаткові приховані тести до того самого публічного контракту.

Локальний шлях заготовки викладача — `git/Lab Assignment 1`. Фіксованими тестами викладача є тести у `Lab1PublicTest.java`. Дозволено редагувати такі файли: `submission.properties`, `Main.java`, `README.md`, створювати предметний клас свого варіанта, власний клас тестів і файл `docs/report.pdf`. Заборонено змінювати, видаляти або перейменовувати файл фіксованих тестів, а також вимикати чи обходити тести через `@Disabled`, налаштування Maven або інші засоби. Початковий запуск тестів виконують до реалізації, а перед здачею всі тести мають проходити успішно. Викладач перевіряє незмінність тестів, порівнюючи їхню контрольну суму, та повторно запускає поданий код з оригінальними тестами. Якщо фіксовані тести змінено, робота не приймається.

Пояснений робочий приклад

Приклад обчислює площу й периметр прямокутника та визначає, чи є він квадратом. Він демонструє локальні змінні, поля, конструктор, методи, створення об'єкта, введення, перевірку та форматований вивід.

Крок 1. Налаштуйте Maven-проект

Дія. Відкрийте файл `pom.xml` у вузлі `Project Files` Apache NetBeans і задайте наведені координати, версії мови, бібліотеки тестування та модулі складання:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>ua.khnu.kpp</groupId>
  <artifactId>lab-01-example</artifactId>
  <version>1.0.0</version>
```

У тому самому файлі після елемента `version` додайте властивості проекту:

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.release>21</maven.compiler.release>
  <junit.version>6.1.3</junit.version>
</properties>
```

Значення `maven.compiler.release` залиште рівним 21: воно обмежує синтаксис та API навчального коду рівнем Java 21.

Після елемента `properties` додайте залежність JUnit:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Після елемента `dependencies` додайте конфігурацію плагінів і закрийте кореневий елемент `project`:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.15.0</version>
      <configuration>
        <release>${maven.compiler.release}</release>
      </configuration>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.5</version>
    </plugin>
  </plugins>
</build>
</project>
```

Очікуваний результат. NetBeans розпізнає Maven-проект без позначок помилки. **Перевірка.** Виконайте `Reload Project`; у вікні `Output` не повинно бути повідомлень про некоректний POM.

Крок 2. Опишіть предметний клас

Дія. У каталозі `src/main/java` створіть у пакеті `ua.khnu.kpp.lab1` файл `Rectangle.java` у UTF-8:

```
1 package ua.khnu.kpp.lab1;
2
3 public final class Rectangle {
```

Оголосіть поля `final` і додайте конструктор. Перевірки виконайте до присвоєння: помилкові розміри не повинні зберігатися в полях.

```
1     private final double width;
2     private final double height;
3
```

```
4 public Rectangle(double width, double height) {  
5     if (!Double.isFinite(width) || width <= 0  
6         || !Double.isFinite(height) || height <= 0) {  
7         throw new IllegalArgumentException(  
8             "Dimensions must be finite and positive");  
9     }  
10    this.width = width;  
11    this.height = height;  
12 }
```

Після конструктора додайте обчислювальні методи й закрийте клас:

```
1 public double area() {  
2     return width * height;  
3 }  
4  
5 public double perimeter() {  
6     return 2 * (width + height);  
7 }  
8  
9 public boolean isSquare() {  
10    return Double.compare(width, height) == 0;  
11 }  
12 }
```

Очікуваний результат. Клас має два незмінні поля, конструктор із перевіркою інваріанта та три методи без консольного введення або виведення. **Перевірка.** Ім'я публічного класу точно збігається з іменем файла; перший рядок визначає пакет `ua.khnu.kpp.lab1`.

Як працює код. Назви параметрів збігаються з назвами полів: `this.width` означає поле, а `width` — параметр. Методи повертають необокруглені результати; `Double.compare` порівнює розміри, а `Double.isFinite` відхиляє NaN та нескінченність, тому недопустимий об'єкт не створюється.

Крок 3. Додайте точку входу

Дія. У тому самому пакеті створіть файл `src/main/java/ua/khnu/kpp/lab1/Main.java` і вставте код:

```
1 package ua.khnu.kpp.lab1;  
2  
3 import java.util.Locale;  
4 import java.util.Scanner;
```

Оголосіть клас, константу `ERROR` та закритий конструктор, який забороняє створення об'єкта `Main`:

```
1 public final class Main {  
2     private static final String ERROR = "ERROR";  
3  
4     private Main() {  
5     }  
6 }
```

Далі додайте метод `main`, який читає рівно два числові значення та відхиляє неповне або надлишкове введення:

```
1 public static void main(String[] args) {
2     Scanner input = new Scanner(System.in)
3         .useLocale(Locale.ROOT);
4     if (!input.hasNextDouble()) {
5         System.out.println(ERROR);
6         return;
7     }
8     double width = input.nextDouble();
9
10    if (!input.hasNextDouble()) {
11        System.out.println(ERROR);
12        return;
13    }
14    double height = input.nextDouble();
15
16    if (input.hasNext()) {
17        System.out.println(ERROR);
18        return;
19    }
```

Після перевірки введення створіть об'єкт, надрукуйте результати й закрийте метод та клас:

```
1     try {
2         Rectangle rectangle = new Rectangle(width, height);
3         System.out.printf(Locale.ROOT,
4             "RESULT_1=%.2f%n", rectangle.area());
5         System.out.printf(Locale.ROOT,
6             "RESULT_2=%.2f%n", rectangle.perimeter());
7     } catch (IllegalArgumentException exception) {
8         System.out.println(ERROR);
9     }
10 }
11 }
```

Очікуваний результат. `Main` читає рівно два числові токени, створює `Rectangle` і друкує лише рядки формалізованого протоколу. Неповне, надлишкове, нечислове або недопустиме введення дає один рядок `ERROR`. **Перевірка.** У файлі немає формул площі чи периметра; вивід не містить запрошень, пояснень або трасування винятку.

Як працює код. `Scanner` розбиває потік введення на токени. Використання `Locale.ROOT` однозначно задає крапку як десятковий роздільник. Метод `hasNextDouble()` перевіряє токен до читання, а `input.hasNext()` після другого числа виявляє надлишкові дані. Формат `%.2f%n` показує дві цифри після крапки та переносить рядок.

Крок 4. Реалізуйте модульні тести

Дія. У пакеті `ua.khnu.kpp.lab1` каталогу `src/test/java` створіть файл `RectangleTest.java`:

```
1 package ua.khnu.kpp.lab1;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import static org.junit.jupiter.api.Assertions.assertFalse;
5 import static org.junit.jupiter.api.Assertions.assertThrows;
6 import static org.junit.jupiter.api.Assertions.assertTrue;
7
8 import org.junit.jupiter.api.Test;
```

Далі оголосіть тестовий клас із двома позитивними сценаріями:

```
1 class RectangleTest {
2     private static final double EPSILON = 1e-9;
3
4     @Test
5     void calculatesControlValues() {
6         Rectangle rectangle = new Rectangle(4.5, 3.0);
7
8         assertEquals(13.5, rectangle.area(), EPSILON);
9         assertEquals(15.0, rectangle.perimeter(), EPSILON);
10        assertFalse(rectangle.isSquare());
11    }
12
13    @Test
14    void recognizesSquare() {
15        Rectangle rectangle = new Rectangle(2.0, 2.0);
16
17        assertEquals(4.0, rectangle.area(), EPSILON);
18        assertEquals(8.0, rectangle.perimeter(), EPSILON);
19        assertTrue(rectangle.isSquare());
20    }
21 }
```

Завершіть клас негативним тестом параметрів конструктора:

```
1 @Test
2 void rejectsInvalidDimensions() {
3     assertThrows(IllegalArgumentException.class,
4         () -> new Rectangle(0.0, 2.0));
5     assertThrows(IllegalArgumentException.class,
6         () -> new Rectangle(Double.NaN, 2.0));
7 }
8 }
```

Очікуваний результат. Три тести незалежно перевіряють контрольні значення, альтернативний допустимий випадок і відхилення недопустимих параметрів. **Перевірка.** NetBeans відображає клас у вузлі Test Packages; імпорти JUnit не позначено як помилкові.

Як працює код. Метод `assertEquals` порівнює дробові результати з допуском 10^{-9} . Метод `assertThrows` підтверджує тип винятку, що становить частину публічного контракту конструктора.

Крок 5. Налаштуйте склад репозиторію

Дія. Створіть у корені файл `.gitignore`:

```
target/  
*.class  
*.log  
nbproject/private/  
.DS_Store
```

У файлі `submission.properties` запишіть `lab=1` і `variant=0`; нуль позначає референсний приклад.

Очікуваний результат. Результати складання й локальні параметри середовища не входять до версії проєкту. **Перевірка.** Після складання команда `git status --short --ignored` повинна позначати `target/` як проігнорований каталог.

Крок 6. Виконайте відтворюване складання

Дія. Відкрийте термінал у корені `lab-01-example` та виконайте одну з команд:

```
# Windows PowerShell або Command Prompt  
.\mvnw.cmd -B -ntp clean verify  
  
# Linux або macOS  
./mvnw -B -ntp clean verify
```

У NetBeans цьому відповідають команди `Clean and Build`, а потім `Test` у контекстному меню проєкту.

Очікуваний результат. Maven видаляє попередній каталог `target`, компілює код із рівнем Java 21 і виконує три тести. **Перевірка.** Наприкінці журналу мають бути відсутні відмови й помилки тестів та наявний статус `BUILD SUCCESS`; кількість пропущених тестів дорівнює нулю.

Крок 7. Перевірте консольний протокол

Дія. Після успішного складання виконайте:

```
java -cp target/classes ua.khnu.kpp.lab1.Main
```

Введіть числа 4.5 і 3, розділивши їх пробілом або переносом рядка.

```
RESULT_1=13.50  
RESULT_2=15.00
```

Очікуваний результат. Площа дорівнює 13.5, а периметр — 15. **Перевірка.** Програма друкує рівно два наведені рядки; назви, порядок, десяткова крапка та кількість знаків збігаються.

Крок 8. Виконайте негативні та граничні перевірки

Дія. Повторно запускайте клас `Main` командою з попереднього кроку й використайте дані з таблиці.

Табл. 4: Контрольні тести прикладу

№	Введення	Очікуваний результат і перевірка
1	2 2	Рівно <code>RESULT_1=4.00</code> і <code>RESULT_2=8.00</code> .
2	0 3	Рівно один рядок <code>ERROR</code> ; числових результатів немає.
3	abc	Рівно один рядок <code>ERROR</code> ; трасування винятку немає.
4	4.5	Неповне введення дає рівно один рядок <code>ERROR</code> .
5	4.5 3 7	Надлишковий токен дає рівно один рядок <code>ERROR</code> .

Очікуваний результат. Допустимі дані обчислюються, а кожна категорія некоректного введення відхиляється однаково. **Перевірка.** Зіставте стандартний вивід із таблицею; будь-який додатковий текст означає порушення консольного контракту.

Порядок виконання

Індивідуальну частину виконайте послідовно; кожен крок має власний артефакт, очікуваний результат і спосіб перевірки.

1. Визначте номер варіанта за формулою номер залікової книжки $\text{mod } 12 + 1$ або отримайте його від викладача. Запишіть номер у звіт і звірте назву задачі зі списком варіантів.
2. У файлі `submission.properties` замініть `NN` на номер варіанта. Перевірте, що файл містить рівно два параметри: `lab=1` і `variant=NN`.
3. Скопіюйте або клонуйте надану заготовку `Lab Assignment 1` у власний робочий каталог і відкрийте її в NetBeans як наявний Maven-проект. Не вилучайте `pom.xml`, `Maven Wrapper`, `.gitignore` та `Lab1PublicTest.java`; команда `mvnw.cmd --version` має показати Maven 3.9.16.
4. До внесення реалізації виконайте `mvnw.cmd -B -ntp clean test`. Очікується успішна компіляція проекту та невдалий тест із чітким повідомленням про потребу замінити `variant=NN` або створити клас варіанта. Це вихідний стан заготовки, а не результат завершеної роботи.
5. У пакеті `ua.khnu.kpp.lab1` реалізуйте предметний клас свого варіанта з точними назвами конструктора й методів. Методи не читають із консолі, не друкують даних і повертають неокруглені значення.
6. У конструкторі перевірте всі обмеження варіанта. Недопустимі параметри мають спричиняти `IllegalArgumentException`; це підтверджується окремим модульним тестом.
7. У файлі `Main.java` реалізуйте визначений у розділі 4 консольний протокол. Перевірте контрольний, неповний, надлишковий і нечисловий набори; стандартний вивід має точно відповідати контракту.
8. Не змінюючи наданий `Lab1PublicTest.java`, у каталозі `src/test/java` створіть власний клас тестів із трьома або більшою кількістю активних тестів: контрольним, альтернативним допустимим і негативним. Усі порівняння `double` виконуйте з допуском 10^{-9} .
9. Виконайте `mvnw.cmd -B -ntp clean verify` у Windows або `./mvnw -B -ntp clean verify` у Linux чи macOS. Перевірте статус `BUILD SUCCESS` і нульову кількість відмов, помилок та пропущених тестів.
10. Якщо заготовку отримано як звичайну папку, ініціалізуйте репозиторій командами `git init -b main`, `git add .` і `git commit`. Команда `git status` після коміту має повідомляти про відсутність незбережених змін. Якщо заготовку клоновано, використовуйте на-

явний репозиторій і створіть власний віддалений origin.

11. Створіть публічний віддалений репозиторій на GitHub, GitLab, Codeberg або іншій платформі розміщення Git, додайте його як origin і виконайте `git push -u origin main`. Доступ без автентифікації перевірте командою `git ls-remote URL`.
12. Додайте звіт до `docs/report.pdf`, повторно виконайте `clean verify`, створіть завершальний коміт і позначте його тегом `lab-01`. Зафіксуйте результат `git rev-parse HEAD` і надішліть тег командою `git push origin lab-01`.

4. Завдання

Реалізуйте програму за призначеним варіантом. Назви пакетів, класів, конструктора й методів утворюють публічний контракт: автоматизована система використовує їх без попереднього перетворення коду. Референсний приклад визначає структуру й підхід до тестування, але не містить розв'язку індивідуального варіанта.

1. Усі класи розмістіть у пакеті `ua.khnu.kpp.lab1` та в стандартних каталогах Maven.
2. Предметний клас оголосіть публічним; сигнатури його конструктора й методів мають точно відповідати варіанту.
3. Обчислювальні методи повинні повертати `double` без округлення, консольного введення й виведення.
4. Конструктор повинен відхиляти недопустимі параметри винятком `IllegalArgumentException`.
5. Клас `Main` має відповідати лише за читання токенів, створення об'єкта й виведення результатів за визначеним нижче протоколом.
6. У репозиторії мають бути публічні тести JUnit; команда `clean verify` повинна завершуватися успішно з чистої копії проєкту.

Індивідуальні варіанти

Кожен пункт задає назву класу, вхідні величини, потрібні результати та контрольний набір. Контрольні значення призначені для тестування, але не є готовою реалізацією.

1. **Room.** Клас `Room` має конструктор із параметрами `double length`, `double width` і `double height` у зазначеному порядку. Метод `floorArea()` повертає lw , `volume()` — lwh , `wallArea()` — $2(l + w)h$. Контрольні аргументи: 5, 4, 2.8; результати: 20.00, 56.00, 50.40.
2. **Trip.** Клас `Trip` має конструктор із параметрами `double distanceKm`, `double fuelLitres` і `double pricePerLitre` у зазначеному порядку. Методи: `fuelConsumptionPer100Km()`, `totalFuelCost()`, `costPer100Km()`. Контрольні аргументи: 420, 31.5, 59.9; результати: 7.50, 1886.85, 449.25.
3. **Deposit.** Клас `Deposit` має конструктор із параметрами `double principal`, `double annualRatePercent` і `int months` у зазначеному порядку. Методи: `interest()` повертає $Prm / (100 \cdot 12)$, `finalAmount()`, `averageMonthlyIncome()`. Контрольні аргументи: 10000, 8.5, 9; результати: 637.50, 10637.50, 70.83.
4. **Cylinder.** Клас `Cylinder` має конструктор із параметрами `double radius` і `double height` у зазначеному порядку. Метод `volume()` повертає $\pi r^2 h$, `lateralArea()` — $2\pi r h$, `totalArea()` — $2\pi r(r + h)$. Контрольні аргументи: 3, 5; результати: 141.37, 94.25, 150.80.

5. **Parcel.** Клас Parcel має конструктор із параметрами `double weightKg`, `double distanceKm`, `double tariff` і `double insurancePercent` у зазначеному порядку. Методи: `transportCost()` повертає $w(d/100)t$, `insuranceCost()`, `totalCost()`. Контрольні аргументи: 12.5, 320, 4.8, 1.5; результати: 192.00, 2.88, 194.88.
6. **ElectricityUsage.** Клас ElectricityUsage має конструктор із параметрами `double previous`, `double current`, `double tariff` і `int days` у зазначеному порядку. Методи: `consumption()`, `cost()`, `averageDailyConsumption()`. Контрольні аргументи: 12450, 12638, 4.32, 30; результати: 188.00, 812.16, 6.27.
7. **HealthMetrics.** Клас HealthMetrics має конструктор із параметрами `double weightKg`, `double heightM` і `double targetWeightKg` у зазначеному порядку. Методи: `bodyMassIndex()` повертає w/h^2 , `bodySurfaceArea()` — $\sqrt{(100h)w/3600}$, `targetDifference()` — $w - w_t$. Контрольні аргументи: 78, 1.82, 72; результати: 23.55, 1.99, 6.00.
8. **Order.** Клас Order має конструктор із параметрами `unitPrice` типу `double`, `quantity` типу `int` і `discountPercent` типу `double` у зазначеному порядку. Методи: `subtotal()`, `discountAmount()`, `amountDue()`. Контрольні аргументи: 249.90, 3, 10; результати: 749.70, 74.97, 674.73.
9. **Route.** Клас Route має конструктор із параметрами `double distanceKm`, `double speedKmH` і `double stopsMinutes` у зазначеному порядку. Методи: `travelTimeHours()`, `totalTimeHours()`, `actualAverageSpeed()`. Контрольні аргументи: 360, 80, 45; результати: 4.50, 5.25, 68.57.
10. **CourseResult.** Клас CourseResult має конструктор із параметрами `double labAverage`, `double practicalAverage` і `double examScore` у зазначеному порядку. Фіксовані ваги — 35%, 25% і 40%. Методи: `labContribution()`, `practicalContribution()`, `examContribution()`, `finalScore()`, `marginAbovePass()`; поріг — 60. Контрольні аргументи: 82, 90, 76; результати: 28.70, 22.50, 30.40, 81.60, 21.60.
11. **Recipe.** Клас Recipe має конструктор із параметрами `int baseServings`, `int desiredServings`, `double flourGrams` і `double milkMl` у зазначеному порядку. Методи: `scaleFactor()`, `scaledFlourGrams()`, `scaledMilkMilliliters()`. Контрольні аргументи: 4, 10, 300, 500; результати: 2.50, 750.00, 1250.00.
12. **CurrencyPurchase.** Клас CurrencyPurchase має конструктор із параметрами `double amount`, `double exchangeRate` і `double commissionPercent` у зазначеному порядку. Методи: `baseCost()`, `commissionAmount()`, `totalCost()`. Контрольні аргументи: 250, 45.2, 1.2; результати: 11300.00, 135.60, 11435.60.

Обмеження параметрів конструктора

Кожний параметр типу `double` має бути скінченним: значення NaN, додатна й від’ємна нескінченність є недопустимими. Крім цієї загальної умови, застосуйте такі обмеження:

- для `Room`, `Trip` і `Cylinder` усі параметри мають бути більшими за нуль;
- для `Deposit`: `principal > 0`, ставка належить відрізьку $[0; 100]$, `months > 0`;
- для `Parcel`: маса, відстань і тариф більші за нуль, страховий відсоток належить відрізьку $[0; 100]$;
- для `ElectricityUsage`: `previous >= 0`, `current >= previous`, `tariff > 0`, `days > 0`;
- для `HealthMetrics` маса, зріст і цільова маса мають бути більшими за нуль;
- для `Order`: `unitPrice > 0`, `quantity > 0`, знижка належить відрізьку $[0; 100]$;

- для `Route`: відстань і швидкість більші за нуль, тривалість зупинок невід’ємна;
- для `CourseResult` кожний бал належить відрізьку $[0; 100]$;
- для `Recipe`: обидві кількості порцій є додатними цілими числами, маса борошна й об’єм молока більші за нуль;
- для `CurrencyPurchase`: сума й курс більші за нуль, комісія належить відрізьку $[0; 100]$.

Порушення будь-якої умови має спричинити `IllegalArgumentException`. Предметний клас не замінює недопустиме значення значенням за замовчуванням і не друкує повідомлення.

Консольний протокол

Автоматизований запуск передає значення через стандартний потік введення в порядку параметрів конструктора. Токени розділяються пробілами або переносами рядка; дробові числа використовують крапку. Програма не виводить запрошень до введення.

Для допустимого набору `Main` викликає методи в порядку їх наведення у варіанті та друкує кожний результат в окремому рядку:

```
RESULT_1=12.34
RESULT_2=56.78
RESULT_3=90.12
```

Варіант `CourseResult` формує п’ять рядків; інші варіанти — три. Формат кожного рядка — `RESULT_N=%.2f`; застосовують `Locale.ROOT`. За нечислового, неповного, надлишкового або недопустимого введення стандартний вивід повинен містити рівно один рядок `ERROR`. Трасування винятку та додатковий текст не допускаються.

План перевірки

Публічні тести зберігають у `src/test/java/ua/khnu/kpp/lab1`. Клас тестів називають за схемою `НазваПредметногоКласуTest`. Мінімальний комплект містить:

1. тест контрольного набору з опису варіанта;
2. тест іншого допустимого набору, очікування для якого попередньо обчислено незалежно від реалізованих методів;
3. тест недопустимих параметрів із перевіркою `IllegalArgumentException`.

Для порівняння неокруглених результатів `double` використовуйте допуск 10^{-9} . Тести не повинні залежати від порядку запуску, мережі, поточного часу або локальних файлів. Анотації `@Disabled` і умовне пропускання тестів не застосовують.

Після клонування система викладача може додати приховані тести. Вони перевіряють той самий оприлюднений контракт: альтернативні допустимі значення, межі діапазонів, `NaN`, нескінченність і консольний протокол. Приховані тести не висувають вимог, яких немає в цій роботі.

Критерії автоматизованого приймання

Автоматизована перевірка послідовно контролює доступність подання, відтворюваність складання, результати тестів і склад репозиторію.

Табл. 5: Тести попереднього технічного контролю

Код	Об'єкт перевірки	Умова успішного виконання
A01	Доступність репозиторію	Публічний URL клонується через HTTPS без автентифікації; поданий коміт або тег lab-01 існує.
A02	Структура й метадані	Наявні pom.xml, Maven Wrapper, стандартні каталоги src/main/java і src/test/java; значення submission.properties відповідають роботі та варіанту.
A03	Відтворюване складання	Команда mvnw.cmd -B -ntp clean verify або її POSIX-відповідник повертає код 0; кількість невдалих, помилкових і пропущених тестів — 0.
A04	Контрольний набір	Публічні JUnit-тести підтверджують усі результати варіанта з допуском 10^{-9} .
A05	Альтернативні значення	Приховані тести створюють предметний об'єкт з іншими допустимими аргументами; усі методи повертають неокруглені очікувані значення.
A06	Межі й недопустимі параметри	Конструктор приймає допустимі граничні значення та викидає IllegalArgumentException для кожної недопустимої категорії.
A07	Консольний протокол	Допустиме введення формує лише рядки RESULT_N; нечислове, неповне, надлишкове й недопустиме введення формує лише ERROR.
A08	Склад репозиторію	У коміті немає target, файлів .class, кешів, журналів, облікових даних або секретів.

Проходження A01–A08 означає завершення попереднього технічного контролю, але не визначає підсумкову оцінку. Звіт, якість пояснення рішення й захист оцінюються окремо.

Аналіз результатів

Табл. 6: Ознаки завершеної роботи

Результат	Спосіб перевірки
Відтворювана збірка	Клонувати репозиторій у чистий каталог і виконати Maven Wrapper із ціллю clean verify; результат — BUILD SUCCESS.
Правильні формули	Контрольний та альтернативний JUnit-тести збігаються з незалежно обчисленими очікуваннями з допуском 10^{-9} .
Визначена реакція на помилки	Конструктор відхиляє недопустимий стан, а Main формує один рядок ERROR без трасування винятку.
Відтворювана версія	Команда git rev-parse HEAD повертає поданий ідентифікатор; git ls-remote URL підтверджує публічний доступ.

Типові помилки та способи виправлення

Діагностуйте помилку за першим повідомленням компілятора або за першим розходженням із контрольним тестом.

Табл. 7: Діагностика типових помилок

Ознака	Ймовірна причина	Виправлення і перевірка
NetBeans запускається не на JDK 25	Середовище використовує іншу платформу Java або встановлено лише JRE.	Перевірте Help > About і Tools > Java Platforms; додайте каталог Temurin JDK 25 та повторно відкрийте проєкт.
Команду mvnw.cmd не знайдено	Термінал відкрито не в корені проєкту або Maven Wrapper не створено.	Перейдіть до каталогу з pom.xml; перевірте наявність mvnw.cmd, mvnw і .mvn/wrapper.
Maven не завантажує залежності	Відсутнє мережеве з'єднання, налаштування проксі некоректне або локальний кеш пошкоджено.	Перевірте мережу й налаштування Maven у NetBeans; повторіть clean verify та зафіксуйте першу помилку завантаження.
class ... is public, should be declared...	Ім'я файлу не збігається з публічним класом.	Перейменуйте файл з урахуванням реєстру; повторіть фазу testCompile.
Прихований тест не знаходить клас	Пакет, ім'я класу, конструктора або методу не відповідає контракту.	Звірте кожну сигнатуру з варіантом і виконайте mvnw.cmd clean verify.
Maven повідомляє No tests to run	Тест збережено поза src/test/java, ім'я файлу не завершується на Test або немає @Test.	Виправте шлях та ім'я; у звіті Surefire має бути щонайменше три активні тести.
expected ... but was ...	Формула, порядок аргументів або очікуване значення тесту є помилковими.	Порівняйте перше розходження з незалежним ручним обчисленням; не отримуйте очікування викликом методу, який перевіряється.
Публічні тести успішні, приховані — ні	Результат округлюється всередині методу, перевірено не всі межі або конструктор приймає NaN чи нескінченність.	Поверніть неокруглене значення, додайте перевірку Double.isFinite та граничні тести.
Перевірка Main не збігається	Програма друкує запрошення, одиниці, зайвий текст або використовує кому.	Залиште лише RESULT_N=%.2f чи ERROR; застосуйте Locale.ROOT і повторіть консольні сценарії.

Ознака	Ймовірна причина	Виправлення і перевірка
<code>git ls-remote URL</code> повертає помилку	Репозиторій приватний, URL некоректний або коміт не надіслано.	Увімкніть публічний доступ, перевірте HTTPS-адресу та виконайте <code>git push</code> ; повторіть перевірку без автентифікації.
У коміті є <code>target</code> або секрет	Файл додано до налаштування <code>.gitignore</code> або перевірку <code>git status</code> пропущено.	Вилучіть артефакт з індексу, за потреби відкличте секрет, доповніть <code>.gitignore</code> і створіть новий коміт.

Вимоги до звіту

Звіт має бути достатнім для відтворення роботи та містити:

1. титульну сторінку із назвою роботи;
2. тему, мету, вимірювані результати навчання й номер варіанта;
3. таблицю вхідних даних, типів, одиниць вимірювання та допустимих діапазонів;
4. математичні формули варіанта і пояснення порядку обчислень;
5. структуру Maven-проєкту та призначення основних файлів;
6. вихідний код предметного класу й Main;
7. вихідний код публічних тестів JUnit;
8. версії JDK, Apache NetBeans, Maven і JUnit та команду `clean verify`;
9. таблицю контрольного, альтернативного й негативного тестів з очікуваними та фактичними результатами;
10. URL публічного репозиторію, повний ідентифікатор коміту й тег `lab-01`;
11. висновок про досягнення результатів навчання та інтерпретацію результатів тестування.

Знімки екрана можуть доповнювати, але не замінювати текст команд, журнал тестів і таблицю результатів.

Захист

На захисті студент клонує поданий коміт у чистий каталог, виконує Maven Wrapper, демонструє контрольний і запропонований викладачем тест, пояснює вибір типів, перевірки конструктора, формули та структуру тестів. Студент також прогнозує результат зміни одного вхідного параметра. Якщо тест завершується невдало, необхідно визначити перше розходження й обґрунтувати спосіб виправлення.

Здача лабораторної роботи через Git

Лабораторна робота допускається до захисту та вважається поданою лише після завантаження проєкту до публічного віддаленого Git-репозиторію й надання викладачеві доступу до зафіксованого коміту. Оцінюється саме зазначений коміт; пізніші зміни не замінюють його без погодженого повторного подання. Виконайте таку послідовність:

1. Збережіть історію, що відображає етапи розроблення. Створіть щонайменше три змістовні коміти, наприклад: початкова структура проєкту; реалізація предметного класу

- й Main; тести та звіт. Перед кожним комітом виконайте `git status`, потім `git add .` і `git commit -m "...".`
- Надішліть гілку командою `git push -u origin main`. Отримайте фактичний URL командою `git remote get-url origin`. Публічний доступ викладача має зберігатися до завершення оцінювання.
 - Зафіксуйте повний commit SHA командою `git rev-parse HEAD`, створіть тег `lab-01` і надішліть його командою `git push origin lab-01`.
 - Виконайте контрольне клонування в новий каталог командою `git clone URL krp-lab-01-check`, а потім перейдіть на поданий коміт командою `git checkout SHA`. Для складання й тестування виконайте `mvnw.cmd -B -ntp clean verify`; для контрольного запуску використайте команду з `README.md`.
 - Перевірте, що `README.md` містить номер і тему роботи, номер варіанта, версії ПЗ, необхідні знання й початкові умови, точні команди складання, тестування та запуску з чистої папки, а також очікуваний результат. Перевірте `.gitignore`. До репозиторію не додають `target/`, скомпільовані файли, кеші, паролі, токени, ключі або інші секрети.

Що подати

Подайте URL публічного Git-репозиторію, назву гілки `main`, тег `lab-01`, повний ідентифікатор завершального коміту та підтвердження, що коміт і тег завантажені на сервер і доступні викладачеві. Перед поданням перевірте склад репозиторію:

- `pom.xml`, `mvnw`, `mvnw.cmd` і каталог `.mvn/wrapper`;
- `src/main/java` з `Main.java` та предметним класом;
- `src/test/java` з незміненим `Lab1PublicTest.java` і власним класом тестів `JUnit`;
- `submission.properties`, `.gitignore` і `README.md` з реквізитами роботи, початковими умовами, командами складання, тестування й запуску та очікуваним результатом;
- `docs/report.pdf` із завершеним звітом.

Архів не подається. У репозиторії не повинно бути каталогів `target`, файлів `.class`, кешів, паролів, токенів або інших секретних даних.

Критерії оцінювання

Максимальна оцінка становить 8 балів. Прохідний рівень — 60%, тобто 4.8 бала. Автоматизований контроль визначає технічний стан подання, а бали нараховують за наведеними компонентами після перевірки звіту й захисту.

Табл. 8: Рубрика оцінювання

Компонент	Макс.	Повний бал	Частковий бал
Структура Maven і збірка	1.0	Стандартна структура, зафіксовані версії та успішний <code>clean verify</code> з чистої копії.	0.5: відтворення потребує одного виправлення; 0: проєкт не збирається.
Предметний клас і формули	2.5	Сигнатури відповідають контексту, типи доречні, усі методи повертають правильні неокруглені результати.	1.0–2.0: частина моделі або формул має помилки; 0: основну поведінку не реалізовано.

Компонент	Макс.	Повний бал	Частковий бал
Валідація і консольний протокол	1.0	Усі обмеження перевірено; допустимий та помилковий вивід точно відповідає протоколу.	0.5: одна категорія перевірок або форматування неповна; 0: контракт порушено істотно.
Автоматизовані тести	1.5	Є контрольний, незалежний альтернативний і негативний тести; усі активні та перевіряють змістовні твердження.	0.5–1.0: комплект або незалежність очікувань неповні; 0: дієвих тестів немає.
Публічний репозиторій і звіт	1.0	Подані URL, коміт і тег доступні; склад репозиторію та звіт повні, без зайвих артефактів і секретів.	0.5: бракує одного елемента або звіт неповний; 0: репозиторій недоступний чи звіту немає.
Захист	1.0	Студент відтворює збірку, пояснює рішення й тести та виконує додаткову перевірку.	0.5: пояснення часткове; 0: розуміння реалізації не підтверджено.

Табл. 9: Інтерпретація підсумкового результату

Частка максимуму	Орієнтовний діапазон	Інтерпретація
80–100%	6.4–8.0 бала	Результати навчання досягнуто повністю; рішення й тести обґрунтовано.
60–79%	4.8–6.39 бала	Мінімальні результати навчання досягнуто; наявні окремі недоліки.
40–59%	3.2–4.79 бала	Роботу не зараховано; потрібне виправлення суттєвих недоліків.
0–39%	0–3.19 бала	Роботу не зараховано; основні результати навчання не підтверджено.

Джерела

Технічні твердження та команди перевірено за офіційною документацією. Посилання зафіксовано на дату підготовки матеріалу.

1. Oracle. *Java SE Support Roadmap*.
2. Oracle. *JDK 25 Documentation*.
3. Dev.java. *Creating Variables and Naming Them*.
4. Dev.java. *Primitive Types*.
5. Dev.java. *Creating Classes*.
6. Dev.java. *Defining Methods*.
7. Dev.java. *Creating and Using Objects*.
8. Oracle. *The javac Command, JDK 25*.
9. Oracle. *The java Command, JDK 25*.
10. Oracle. *Scanner API, Java SE 25*.

11. Apache NetBeans. *Apache NetBeans 31*.
12. Apache NetBeans. *Java Quick Start Tutorial*.
13. Apache NetBeans. *Using Git Support*.
14. Apache Maven. *Downloading Apache Maven 3.9.16*.
15. Apache Maven. *Introduction to the Standard Directory Layout*.
16. Apache Maven. *Maven Wrapper*.
17. Apache Maven. *Maven Compiler Plugin 3.15.0*.
18. Apache Maven. *Maven Surefire Plugin 3.5.5*.
19. JUnit Team. *JUnit 6.1.3 User Guide*.
20. Git Project. *git-init Documentation*.
21. GitHub Docs. *About Repositories*.
22. Eclipse Adoptium. *Eclipse Temurin Releases*.
23. Eclipse Adoptium. *Installation*.
24. Apache Maven. *What is Maven?*.
25. Apache Maven. *Introduction to the Build Lifecycle*.
26. Gradle. *What is Gradle?*.
27. Apache Ant. *Apache Ant Manual*.

5. Контрольні запитання для самоперевірки

Дайте короткі самостійні відповіді та підготуйтеся пояснити їх на прикладі власного коду.

1. Які ролі виконують JDK, `javac`, байткод і JVM?
2. Що таке Maven, які операції він автоматизує та чим його модель роботи відрізняється від Gradle, Apache Ant і прямих команд `javac/java`?
3. Чому проєкт використовує стандартні каталоги Maven, які параметри задає `pom.xml` і для чого потрібний Maven Wrapper?
4. Що виконують фази Maven `clean` і `verify`?
5. Чим відрізняються `.java` і `.class` та чому Java називають статично типізованою мовою?
6. Чим поле класу відрізняється від локальної змінної та параметра?
7. Які вісім примітивних типів визначає Java і чому `String` не належить до них; чому `5 / 2` і `5.0 / 2` дають різні результати?
8. З яких частин складається оголошення методу та чим параметр відрізняється від аргументу?
9. Яке призначення конструктора, оператора `new` і модифікатора `private`?
10. Чому предметні методи повертають неокруглений `double`, а JUnit-порівняння використовують допуск?
11. Чим відрізняються публічні тести студента від прихованих тестів системи перевірки та які вимоги для них є спільними?
12. Які категорії недопустимих значень повинен перевіряти конструктор предметного класу?
13. Чому консольна програма не повинна друкувати запрошення до введення під час автоматизованої перевірки?
14. Чим Git відрізняється від GitHub, GitLab або Codeberg і для чого під час подання вказують повний ідентифікатор коміту та тег `lab-01`?